SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI-602105

SMART IRRIGATION SYSTEM SIMULATOR USING IOT SENSOR DATA FOR WATER OPTIMIZATION

AGILE SPRINT PLANNING & USER STORY WORKSHOP

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING

under the course

SOFTWARE ENGINEERING

Submitted by
SHAIK KHAJAH ROSHAN (192511445)

Under the Supervision
K ANITA DAVAMANI

JULY-2026
SIMATS SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI-602105

TABLE OF CONTENTS

CHAPTER 1: PROJECT VISION AND STAKEHOLDERS

1.1 Project Vision

To design and develop an intelligent, IoT-enabled Smart Irrigation System Simulator that empowers farmers to make data-driven irrigation decisions in real time. The platform continuously senses field and weather conditions, automatically computes optimal irrigation schedules, and presents actionable insights through interactive dashboards — ultimately minimizing water wastage, reducing operational costs, and improving crop yield and long-term agricultural sustainability.

1.2 Project Objectives

- Continuously collect and analyze real-time environmental data — soil moisture, temperature, humidity, rainfall, and weather forecasts — from field-deployed IoT sensors.
- Automatically determine optimal irrigation duration and frequency using a rule-based / data-driven decision engine tailored to crop and soil conditions.
- Provide farmers with intuitive, interactive dashboards visualizing water consumption trends and crop health indicators.
- Deliver timely alerts regarding upcoming irrigation requirements and abnormal environmental conditions (e.g., sensor failure, extreme heat).
- Support configurable crop types and irrigation policies to accommodate diverse farming scenarios and infrastructure (drip, sprinkler, flood irrigation).
- Minimize water wastage while improving agricultural productivity, cost-efficiency, and environmental sustainability.

1.3 Key Stakeholders

The table below identifies all stakeholders involved in the project along with their roles and interests in the system.

Stakeholder	Role / Interest in the System
Farmers / Growers	Primary end users — operate the system, receive alerts, and configure crop and irrigation settings for their fields.
Product Owner	Represents business and farmer priorities; owns and prioritizes the product backlog.
Scrum Master	Facilitates Agile ceremonies, removes impediments, and ensures adherence to Scrum practices.
Development Team	Backend, frontend, IoT-integration, and QA engineers who design, build, and test the simulator.
Data Scientists / ML Engineers	Design the irrigation decision logic and predictive models using historical and sensor data.
Agronomists / Agri-Consultants	Validate crop-specific irrigation thresholds and ensure agronomic accuracy of recommendations.
IoT Hardware Vendors	Supply and maintain soil-moisture, temperature, humidity, and rainfall sensors deployed in the field.
System Administrators / DevOps	Manage cloud infrastructure, deployment pipelines, data security, and system uptime.

Government / Agri-Extension Bodies	Set sustainability and water-usage compliance standards; may consume aggregate reports.
Investors / Agribusiness Partners	Fund the project and expect measurable ROI, scalability, and market adoption.

Table 1: Key Stakeholders and Their Roles in the Smart Irrigation System Simulator

1.4 System Overview

The diagram below illustrates the high-level architecture of the Smart Irrigation System Simulator, showing the flow of data from field sensors through the decision engine to the dashboards, alerts, and irrigation actuation.

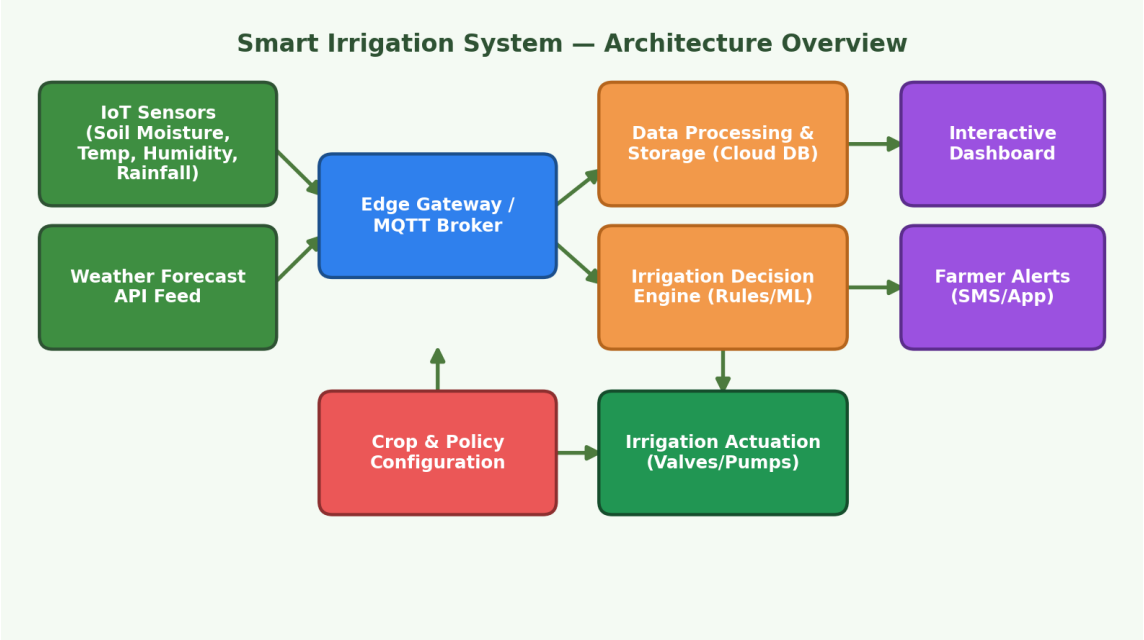

Figure 1: High-level system architecture of the Smart Irrigation System Simulator

CHAPTER 2: EPICS AND USER STORIES

Based on the problem statement, the requirements have been decomposed into six Epics, each broken down into detailed User Stories using the standard Agile format: “As a <user>, I want <feature>, so that <benefit>.”

Epic 1: Sensor Data Acquisition and Integration

- **US1.1:** As the system, I want to continuously collect soil moisture, temperature, and humidity readings from field sensors, so that irrigation decisions are based on accurate, real-time conditions.
- **US1.2:** As a system administrator, I want to integrate third-party weather forecast APIs, so that rainfall predictions can be factored into irrigation planning.
- **US1.3:** As a developer, I want sensor data to be validated and cleaned before storage, so that erroneous or faulty readings do not distort irrigation decisions.

Epic 2: Irrigation Decision Engine (Automation)

- **US2.1:** As a farmer, I want the system to automatically calculate optimal irrigation duration and frequency, so that my crops receive adequate water without manual calculation.
- **US2.2:** As a farmer, I want the system to automatically reduce or skip scheduled irrigation when rainfall is forecasted, so that water is not wasted.
- **US2.3:** As an agronomist, I want irrigation logic to be customizable per crop type and growth stage, so that recommendations remain agronomically accurate.

Epic 3: Dashboard and Visualization

- **US3.1:** As a farmer, I want to view an interactive dashboard of water consumption trends, so that I can track usage over time and identify inefficiencies.
- **US3.2:** As a farmer, I want to visualize crop health indicators on a chart or field map, so that I can quickly identify areas needing attention.
- **US3.3:** As a farm manager, I want to compare water usage across multiple fields, so that I can optimize resource allocation across the farm.

Epic 4: Alerts and Notification System

- **US4.1:** As a farmer, I want to receive real-time alerts when irrigation is due, so that I do not miss critical watering schedules.
- **US4.2:** As a farmer, I want to be notified of abnormal environmental conditions such as extreme heat or sensor failure, so that I can respond promptly.
- **US4.3:** As a farmer, I want to receive alerts through SMS as well as mobile app push notifications, so that I stay informed even with limited internet connectivity.

Epic 5: Crop and Policy Configuration Management

- **US5.1:** As a farmer, I want to configure crop types and their water requirements, so that irrigation schedules match crop-specific needs.

- **US5.2:** As a farm manager, I want to define custom irrigation policies for different scenarios such as drip or sprinkler systems, so that the system adapts to my existing infrastructure.
- **US5.3:** As an administrator, I want to manage user roles and permissions, so that only authorized personnel can modify critical configurations.

Epic 6: Water Usage Analytics and Reporting

- **US6.1:** As a farm manager, I want to generate weekly and monthly water usage reports, so that I can evaluate irrigation efficiency and cost savings.
- **US6.2:** As a sustainability officer, I want to track water savings compared to traditional irrigation methods, so that I can report on environmental impact.

CHAPTER 3: ACCEPTANCE CRITERIA

Clear, measurable acceptance criteria have been defined for the highest-priority user story in each Epic using the Given–When–Then format, ensuring each story is testable and unambiguous.

US1.1 — Real-time Sensor Data Collection

Given: the IoT sensors are powered on and connected to the network

When: soil moisture, temperature, and humidity readings are captured at scheduled intervals

Then: the readings are transmitted to the cloud database within 5 seconds and stored with an accurate timestamp

Given: a sensor reading falls outside the expected physical range (e.g., moisture greater than 100%)

When: the data-validation module processes the reading

Then: the reading is flagged as anomalous, logged, and excluded from irrigation calculations

US2.1 — Automatic Irrigation Scheduling

Given: current soil moisture, temperature, humidity, and crop water-requirement data are available

When: the decision engine computes the irrigation schedule

Then: it outputs a recommended irrigation duration (in minutes) and frequency (times/day) within 2 minutes of data availability

Given: rainfall is forecast within the next 24 hours with greater than 70% probability

When: the decision engine evaluates the upcoming schedule

Then: it automatically reduces or skips the next scheduled irrigation cycle and logs the reason

US3.1 — Water Consumption Dashboard

Given: water consumption data exists for the selected field and date range

When: the farmer opens the dashboard

Then: a chart of daily/weekly water usage renders within 3 seconds, with drill-down available by field

US4.1 — Irrigation Due Alerts

Given: the irrigation schedule indicates a watering cycle is due within the next 30 minutes

When: the alert engine runs its scheduled check

Then: the farmer receives a push notification and/or SMS containing the field name, scheduled time, and recommended duration

US5.1 — Crop Configuration

Given: a farmer wants to add a new crop profile

When: they enter the crop name, growth stage, and water-requirement parameters

Then: the system saves the configuration and applies it to all future irrigation calculations for the associated field

US6.1 — Water Usage Reporting

Given: water usage data has been logged for at least one full week

When: the farm manager requests a report for a chosen period

Then: the system generates a downloadable report (PDF/CSV) showing total water used, savings versus baseline, and a per-field breakdown

CHAPTER 4: PRODUCT BACKLOG PRIORITIZATION

The product backlog was prioritized using the MoSCoW technique (Must Have, Should Have, Could Have, Won't Have) combined with business value and technical dependency analysis. Foundational stories — sensor data acquisition and validation — were prioritized first since the decision engine, dashboards, and alerts all depend on reliable, clean data. Core automation (irrigation scheduling) and safety-critical alerts followed, ensuring the Minimum Viable Product (MVP) delivers real, actionable value to farmers from Sprint 1 onward.

ID	User Story (Summary)	Priority (MoSCoW)	Business Value	Story Pts	Sprint
US1.1	Real-time sensor data collection	Must Have	High	**8**	Sprint 1
US2.1	Automatic irrigation scheduling	Must Have	High	**13**	Sprint 1
US1.2	Weather forecast API integration	Must Have	High	**5**	Sprint 1
US4.1	Irrigation-due alerts	Must Have	High	**5**	Sprint 1
US1.3	Sensor data validation & cleaning	Must Have	Medium	**5**	Sprint 1
US5.1	Crop configuration management	Must Have	Medium-High	**8**	Sprint 1
US2.2	Rainfall-based schedule adjustment	Should Have	High	**8**	Sprint 2
US3.1	Water consumption dashboard	Should Have	Medium	**8**	Sprint 2
US4.2	Abnormal condition alerts	Should Have	Medium	**5**	Sprint 2
US2.3	Crop-stage specific irrigation logic	Should Have	Medium	**8**	Sprint 2
US3.2	Crop health visualization	Should Have	Medium	**8**	Sprint 2
US5.2	Custom irrigation policies	Should Have	Medium	**5**	Sprint 2
US6.1	Water usage reports	Could Have	Medium	**5**	Sprint 3
US4.3	SMS / app notification channel	Could Have	Low-Medium	**3**	Sprint 3
US3.3	Multi-field water usage comparison	Could Have	Low	**5**	Sprint 3
US5.3	Role-based access control	Could Have	Low	**3**	Sprint 3
US6.2	Sustainability / water-savings tracking	Won't Have (this release)	Low	**5**	Future Release

Table 2: Product Backlog Prioritized Using the MoSCoW Technique with Story Points and Sprint Allocation

4.1 Prioritization Rationale

- Dependency-driven sequencing: Sensor acquisition (Epic 1) must precede the Decision Engine (Epic 2), which in turn feeds Dashboards (Epic 3) and Alerts (Epic 4).
- High business value items — items directly reducing water wastage and preventing crop loss — are scheduled earliest (Must Have).
- Configuration and secondary analytics features (Epics 5 and 6) are scheduled progressively as the core pipeline stabilizes.
- Low-value, low-urgency items (e.g., sustainability tracking dashboards) are deferred to future releases to protect sprint focus.

The chart below visualizes the total story points allocated to each Epic across the backlog, indicating relative implementation effort.

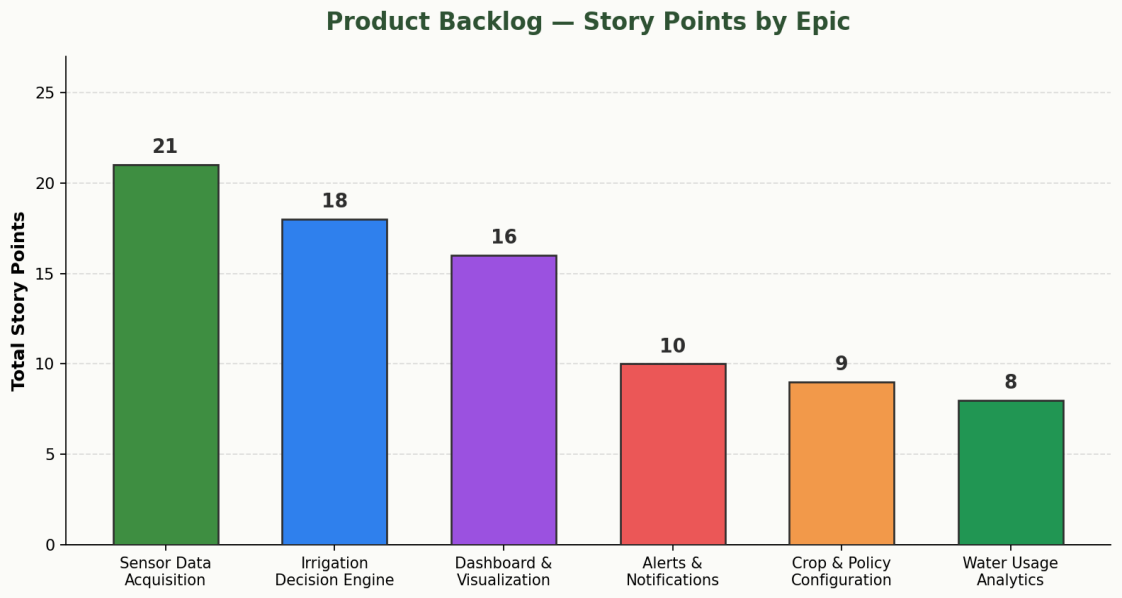

Figure 2: Distribution of story points across Epics in the Product Backlog

CHAPTER 5: SPRINT BACKLOG PLANNING

The Scrum team consists of a Product Owner, a Scrum Master, and a cross-functional Development Team of five members (backend, frontend, IoT integration, data science, and QA). The team follows two-week sprints with Daily Standups (15 minutes), Sprint Planning, Sprint Review, and Sprint Retrospective ceremonies. Based on an assumed team velocity of approximately 40 story points per sprint, the following high-priority (Must Have) stories were selected for Sprint 1.

Sprint 1 Backlog — Selected User Stories

ID	User Story	Story Pts	Owner (Role)
US1.1	Real-time sensor data collection	8	IoT / Backend Engineer
US1.2	Weather forecast API integration	5	Backend Engineer
US1.3	Sensor data validation & cleaning	5	Backend Engineer
US2.1	Automatic irrigation scheduling	13	Data Scientist / Backend
US4.1	Irrigation-due alerts	5	Full-stack Engineer
US5.1	Crop configuration management	8	Frontend + Backend Engineer
Total		**44**	

Table 3: Sprint 1 Backlog — Selected User Stories, Story Points, and Task Owners

Task Breakdown by User Story

US1.1 — Real-time Sensor Data Collection (Tasks)

- Set up MQTT broker and define sensor communication protocol.
- Develop sensor data ingestion microservice (MQTT listener / REST endpoint).
- Configure time-series database schema for sensor readings.
- Write unit and integration tests for the ingestion pipeline.

US1.2 — Weather Forecast API Integration (Tasks)

- Evaluate and select a weather forecast API provider.
- Implement API integration service with response caching.
- Map external forecast data to the internal data model.
- Implement fallback handling for API downtime.

US1.3 — Sensor Data Validation (Tasks)

- Define validation rules and acceptable ranges for each sensor type.
- Implement outlier / anomaly detection logic.
- Build an anomaly log and flagging mechanism.

US2.1 — Automatic Irrigation Scheduling (Tasks)

- Design the irrigation calculation algorithm (rule-based threshold model).
- Implement the decision engine service.

- Integrate soil, weather, and crop-configuration data as inputs.
- Unit test irrigation duration and frequency outputs across scenarios.
- Conduct peer code review and performance testing.

US4.1 — Irrigation-Due Alerts (Tasks)

- Design the alert rule engine and scheduling triggers.
- Integrate a push-notification service (e.g., Firebase Cloud Messaging) and SMS gateway.
- Design notification templates with field name, time, and duration.
- Test end-to-end alert delivery under various network conditions.

US5.1 — Crop Configuration Management (Tasks)

- Design the crop-configuration data model.
- Build the UI form for creating and editing crop profiles.
- Implement backend CRUD APIs for crop profiles.
- Validate configuration persistence and integration with the decision engine.

CHAPTER 6: STORY POINT ESTIMATION

Story points were estimated collaboratively by the Development Team using Planning Poker, with estimates expressed on the Fibonacci sequence (1, 2, 3, 5, 8, 13, 21). The Fibonacci scale was chosen because the increasing gaps between numbers naturally reflect the greater uncertainty associated with larger, more complex stories, discouraging false precision in estimation.

ID	User Story	Points	Justification
US1.1	Sensor data collection	**8**	Moderate complexity integrating multiple sensor types and communication protocols; requires a robust real-time pipeline.
US1.2	Weather API integration	**5**	Well-defined third-party integration with moderate uncertainty around API reliability.
US1.3	Data validation	**5**	Requires designing rule-based anomaly-detection logic and a flagging workflow.
US2.1	Automatic irrigation scheduling	**13**	High complexity — the core decision algorithm combines multiple live data sources and needs extensive testing across crop scenarios.
US4.1	Irrigation-due alerts	**5**	Moderate integration effort connecting scheduling logic to notification services.
US5.1	Crop configuration	**8**	Involves both UI development and backend CRUD work of moderate complexity.

Table 4: Story Point Estimation for Sprint 1 User Stories Using the Fibonacci Scale

Based on these estimates, Sprint 1 totals 44 story points, closely aligned with the team's assumed velocity of approximately 40 points per sprint (a small planned buffer was accepted given the team's growing familiarity with IoT integration work). The projected burndown for Sprint 1 is shown below.

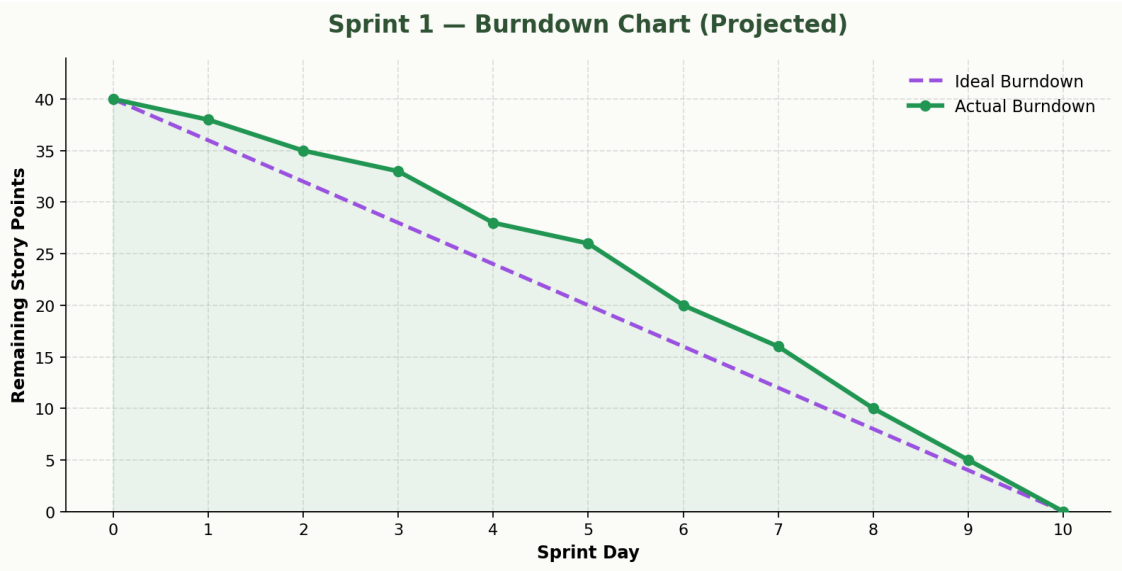

Figure 3: Projected Sprint 1 burndown chart (ideal vs. actual remaining story points)

CHAPTER 7: Sprint Goal and Expected Deliverables

7.1 Sprint 1 Goal

> **“Enable end-to-end automated irrigation scheduling by integrating real-time sensor and weather data, computing optimal irrigation duration and frequency, and alerting farmers of upcoming irrigation needs.”**

7.2 Expected Sprint Deliverables

- A functional sensor-data ingestion pipeline supporting soil moisture, temperature, and humidity sensors.
- A working weather forecast API integration providing rainfall predictions.
- A data validation module that flags anomalous or erroneous sensor readings.
- A functioning irrigation decision engine that outputs recommended duration and frequency.
- Basic crop-profile configuration (create, read, update, delete) accessible to farmers.
- A real-time alert system notifying farmers of upcoming irrigation via app and/or SMS.
- A demo-ready simulation environment showcasing the complete flow — from sensor input, through the decision engine, to farmer alerts.

7.3 Definition of Done

- Code is peer-reviewed, merged, and free of critical defects.
- Unit and integration tests pass with at least 80% coverage for new modules.
- Features are deployed and verified in the staging environment.
- The Sprint Goal is demonstrated to the Product Owner during Sprint Review.
- Relevant technical and user documentation is updated.

7.4 Sprint Ceremonies Summary

Ceremony	Duration	Purpose
Sprint Planning	2 hours	Select backlog items, define the Sprint Goal, and break stories into tasks.
Daily Standup	15 minutes	Synchronize progress, surface blockers, and plan the day's work.
Sprint Review	1 hour	Demonstrate completed work to the Product Owner and stakeholders.
Sprint Retrospective	45 minutes	Reflect on process improvements for the next sprint.

Table 5: Summary of Sprint Ceremonies, Their Duration, and Purpose